

LAB2: 순차논리회로 실습

실험 후 보고서

학과: 전자전기컴퓨터공학부

이름/학번: 정현재/2023440127

작성일: 2026/09/23

대상: 카운터/클록 분주기/레지스터/시프트 레지스터/PISO/무어 머신/밀리 머신/세그먼트 스캔

검증 환경: VS Code VaporView, Vivado 2026.1, FPGA Spartan

보고서 및 실험의 범위

업/다운 카운터, 클록 분주기, 레지스터, 시프트 레지스터, PISO, Moore 상태머신, Mealy 상태 머신 및 7세그먼트 자리 스캔의 총 8개 실험을 대상으로 Vivado 시뮬레이션 및 FPGA 보드 위 검증을 수행하였다. 파형의 관점에서 본 설계는 실험 전 보고서에서 작성하였기 때문에 이 보고서에서는 FPGA 보드 위의 설계를 위주로 설명하겠다.

소스 식별

프로젝트: lab2

기준 커밋: 7468d23823f88e4fd143cf543e1050fca43259c

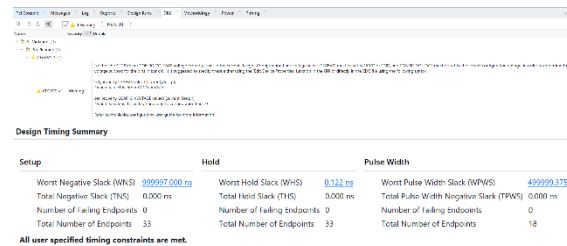
실행본: src/, sim/, constraints/

GitHub 주소: https://github.com/Jeong719/ECE2_Lab2.git

제출 태그: v1.0

화면 하단을 통해 Synthesis,

Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



위 사진은 업/다운 카운터의 DRC 결과 및 timing 보고서다. 해당 사진처럼 DRC 결과 CFGBVS-1 경고 1건이 표시되었다. 이는 FPGA 구성 뱅크의 전압 관련 속성이 지정되지 않았다는 내용이다.

실제 보드 구현 및 동작 분석

실제 보드에서는 rst를 1번 버튼에, enable을 2번 버튼에, down을 dip switch 8번에 할당하였다. Value는 5~8번 LED를 통해 관찰할 수 있었다. down스위치를 off 한 상태에서 2번 버튼을 누르면 LED를 통해 나타나는 4비트 2진수 값이 1씩 오르다가 15 다음에는 0이 되며 순환하는 것을 확인하였고, down 스위치를 on 한 상태에서는 2번 버튼을 누르면 1씩 감소하다가 0 이후에는 15가 되며 순환하는 것을 확인하였다. LED가 나타내는 값이 어떤 값이든 1번 버튼을 누르면 0으로 리셋되는 것을 확인하였다.

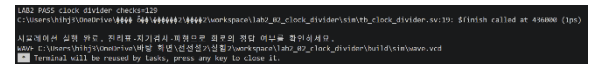
3. 실험2: 클록 분주기

설계와 예상 동작 요약

본 회로는 입력 클록을 각기 다른 divisor로 분주하여 입력 클록 대비 느린 서로 다른 주기의 출력을 생성한다.

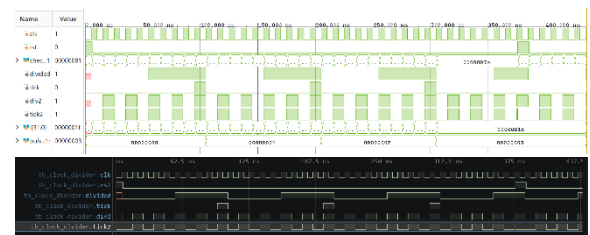
Vivado 시뮬레이션 및 사전 결과 비교

```
# run 1000ns
LAB2_PASS clock_divider checks=129
$finish called at time : 436 ns : Fi
```



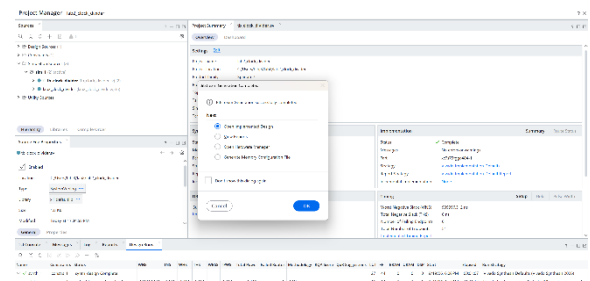
vivado에서 simulation 한 결과

checks=129, 436ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.

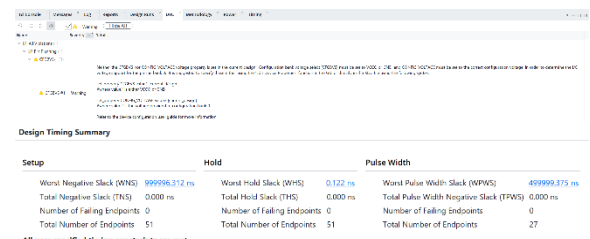


위 사진은 vivado simulation의 파형, 아래는 vscode에서 사전 검증한 파형이다. 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



화면 하단을 통해 Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성

의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

실제 보드에서는 LED 4~8번을 통해 분주 결과를 확인할 수 있었다. 이 때 LED 4번을 tick에 해당되는 LED로 찰나의 순간에만 켜지는 것을 확인하였다. 8번 LED가 입력 주파수를 가장 적게 분주하며 7, 6, 5 LED 순서대로 적게 분주하여 출력 주파수가 낮아지는 것을 확인하였다. 처음에는 10Hz로 시작하여 100Hz부터는 LED 8번의 주기가 너무 짧아 사람의 눈에는 계속 켜져있는 것처럼 보이는 것을 확인하였고 500Hz에서는 7번도 계속 점등되어있는 것처럼 보이는 것을 확인하였다. 이후 1kHz 까지 실험을 진행하였다. 입력 주파수를 올릴 때마다 1번 버튼을 통한 리셋을 눌러 주었다.

4. 실험3: 레지스터

설계와 예상 동작 요약

본 회로는 입력을 저장하는 stored와 stored로부터 저장값을 전달받는 value로 구성된다. Load 스위치가 on인 상태에서 버튼을 누른다면 입력 값은 stored에 저장된다. Transfer 스위치가 on인 상태에서 버튼을 누른다면 갱신 전 stored 값이 value에 전달된다.

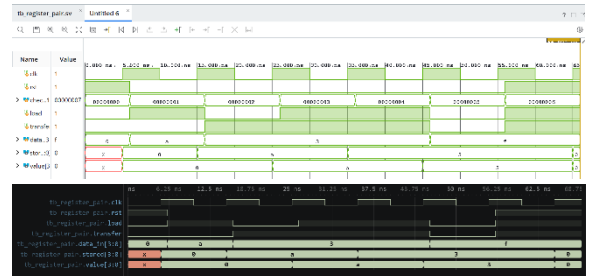
Vivado 시뮬레이션 및 사전 결과 비교

```
LAB2_PASS register_pair checks=7
$finish called at time : 66 ns :
```

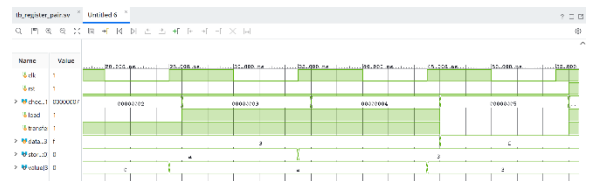
```
LAB2_PASS register_pair checks=7
C:\Users\h131\OneDrive\00000000000000000000\workspace\lab2_03_register\src\register_pair.vv:39: $finish called at 66ns
시뮬레이션 실행 완료. 결과표과 개입사, 파실오로 비로의 정단, 이부분을 확인하십시오.
date: C:\Users\h131\OneDrive\00000000000000000000\workspace\lab2_03_register\src\register_pair\src\register_pair.vv
Terminal will be reused by tasks, press any key to close it.
```

vivado에서 simulation 한 결과 checks=7, 66ns에서 종료된 것을 확인할 수 있다. 이

는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.

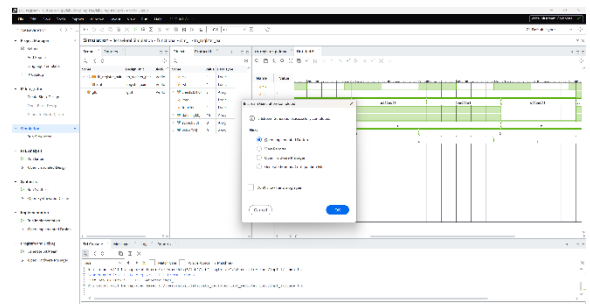


위 사진은 vivado simulation의 파형, 아래는 vscode에서 사전 검증한 파형이다. 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.



Vivado simulation에서도 load=1, transfer=1일 때 value의 값은 data_in이 즉시 전달되는 것이 아닌 이전 stored값이 전달되는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.

Setup	Hold	Pulse Width
Worst Negative Slack (NWS) 999997.812 ns	Worst Hold Slack (NHS) 0.119 ns	Worst Pulse Width Slack (NPPWS) 499999.375 ns
Total Negative Slack (TNS) 0.000 ns	Total Hold Slack (TNS) 0.000 ns	Total Pulse Width Negative Slack (TPNWS) 0.000 ns
Number of Failing Endpoints 0	Number of Failing Endpoints 0	Number of Failing Endpoints 0
Total Number of Endpoints 72	Total Number of Endpoints 72	Total Number of Endpoints 36

All user specified timing constraints are met.

DRC에는 실험 1과 동일한 CFGBVS-1 경고

1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

실제 보드에서는 dip switch 1~4번을 data_in으로 할당하였고, load는 8번, transfer는 7번 dip switch에 할당하였다. 4비트 stored는 1~4번 LED, 4비트 value는 5~8번 LED로 할당하였다. dip switch 1,2,8번을 on 한 상태에서 2번 버튼을 누르니 LED 1,2번이 점등되는 것을 확인하였다. 이후 3,4,7번 스위치만 on한 상태에서 2번 버튼을 누르니 LED 1,2,5,6번이 점등되는 것을 관찰하였고, 이를 통해 load 기능 및 transfer 기능을 확인하였다. 그 상태에서 8번 스위치를 추가적으로 on 한 후 2번 버튼을 눌렀을 때 LED 3,4,5,6이 점등되는 것을 관찰하였고, 이를 통해 갱신 전 stored 값이 value에 전달되는 것을 확인하였다. 7, 8번 스위치를 off 한 상태에서는 아무리 2번 버튼을 눌러도 LED에 변화가 없는 것을 확인하였고, 1번 버튼을 누르니 모든 LED가 소등되는 것을 확인하였다.

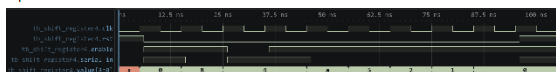
5. 실험4: 시프트 레지스터

설계와 예상 동작 요약

본 회로는 serial_in 스위치의 binary 값을 최상위 비트에 저장하고 기존 비트를 하위 비트 방향으로 이동시키는 4비트 시프트 레지스터다.

Vivado 시뮬레이션 및 사전 결과 비교

```
LAB2_PASS shift_register4 checks=8
$finish called at time : 106 ns : I
```



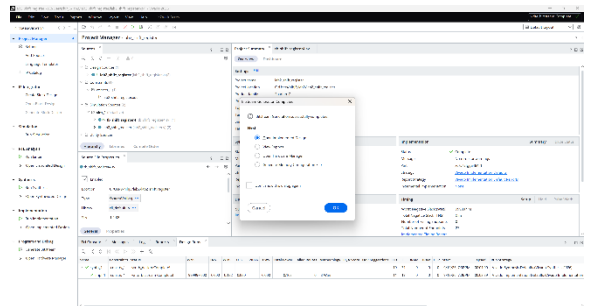
vivado에서 simulation 한 결과 checks=8, 106ns에서 종료된 것을 확인할 수 있다.

이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.



위 사진은 vivado simulation의 파형, 아래는 vscode에서 사전 검증한 파형이다. 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

실제 보드에서는 1번 dip switch에 serial_in을, LED 5~8번에 value 값을 할당

하였다. 1번 스위치의 이진값은 최상위 비트 즉 5번 LED의 값으로 입력됐고 이전 5번 LED에 저장된 비트는 6번, 6번은 7번으로 이동하는 등 오른쪽으로 한 칸씩 이동하는 것을 확인하였다. 이후 1번 버튼을 누르니 모든 LED가 소등되는 것을 관찰하여 rst 버튼이 정상적으로 작동하는 것을 확인하였다.

6. 실험5: PISO

설계와 예상 동작 요약

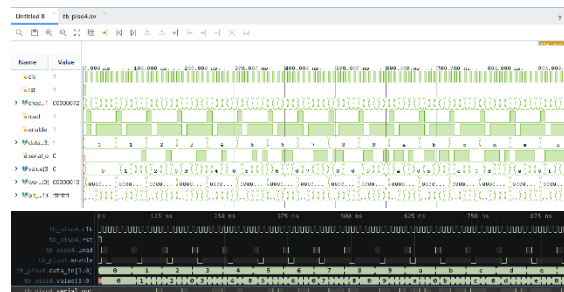
본 회로는 load 스위치를 on으로 했을 때 다른 스위치의 binary 값을 저장하며, 최상위 비트를 출력으로 한다. Load 스위치를 off로 했을 때는 저장값을 왼쪽으로 이동시키며, 최하위 비트는 0으로 채운다.

Vivado 시뮬레이션 및 사전 결과 비교

```
LAB2_PASS piso4 checks=114
$finish called at time : 976 ns
```

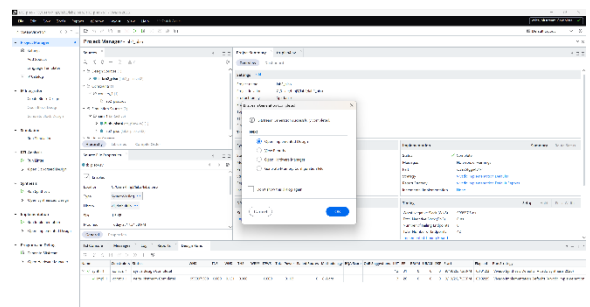
LAB2_PASS piso4 checks=114
C:\Users\min1\OneDrive\0000 000 0000002\workspace\lab2_05 piso\src\th piso4.vv:19: \$finish called at 976000

vivado에서 simulation 한 결과 checks=114, 976ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.



위 사진은 vivado simulation의 파형, 아래는 vscode에서 사전 검증한 파형이다. 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

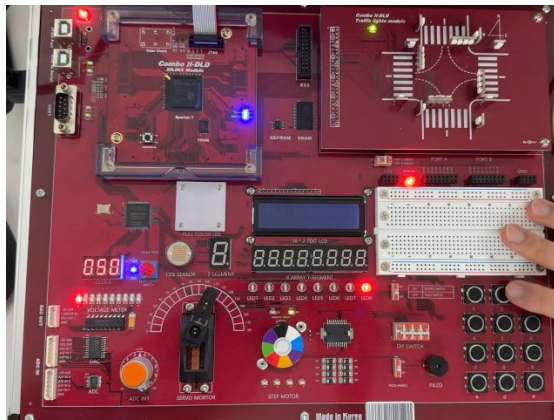
실제 보드 구현 및 동작 분석

실제 보드에서는 dip switch 8번에 load를 할당하였고, 1~4번을 data_in으로 설정하고 LED 1~4번을 value, LED 8번을 serial_out으로 설정하였다. LED 8번은 LED 1번과 늘 같은 상태를 유지한 것을 관찰하였으며, 이를 통해 최상위 비트를 출력으로 하는 serial_out 기능을 정상적으로 확인하였다. 8번 스위치를 on 한 상태에서 2번 버튼을 눌렀을 때 1~4번 스위치의 이진값이 LED 1~4번에 load 되는 것을 확인하였고 이후 8번 스위치를 off한 상태에서 2번 버튼을 누르면 비트가 왼쪽으로 이동하는 것을 확인하였다. 또한 rst가 할당된 1번 버튼을 눌렀을 때는 모든 LED가 소등되는 것을 확인하였다.

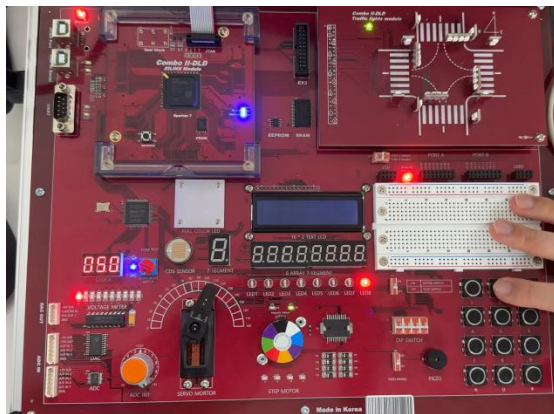
아래는 실제 보드 위에 구현한 사진이다.
각 사진은 enable(2번) 버튼을 누른 직후의

사진이다.

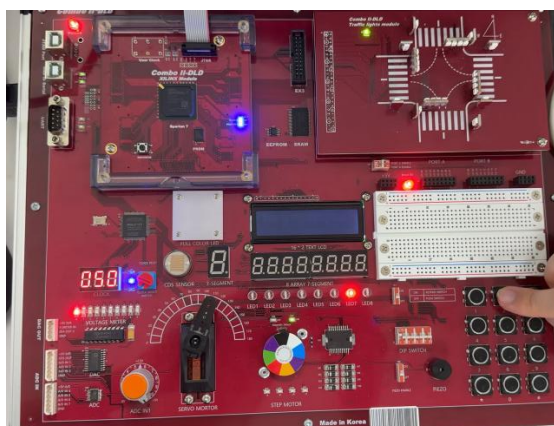
Advance=1, 이전 상태: 00->현재 상태: 01



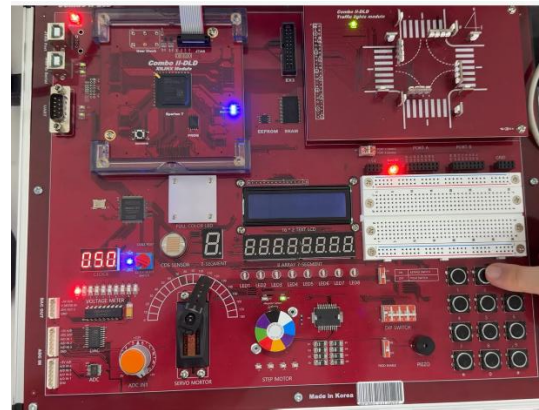
Advance=0, 이전 상태: 01->현재 상태: 01



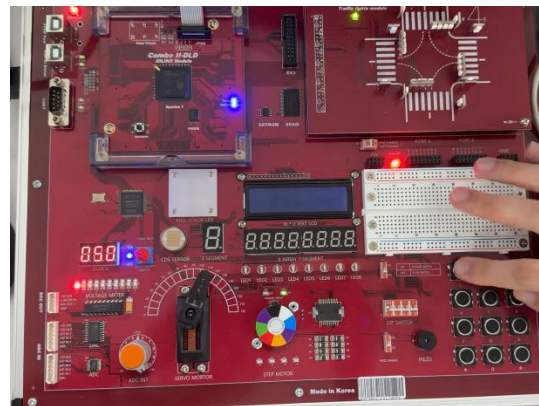
Advance=1, 이전 상태: 01->현재 상태: 10



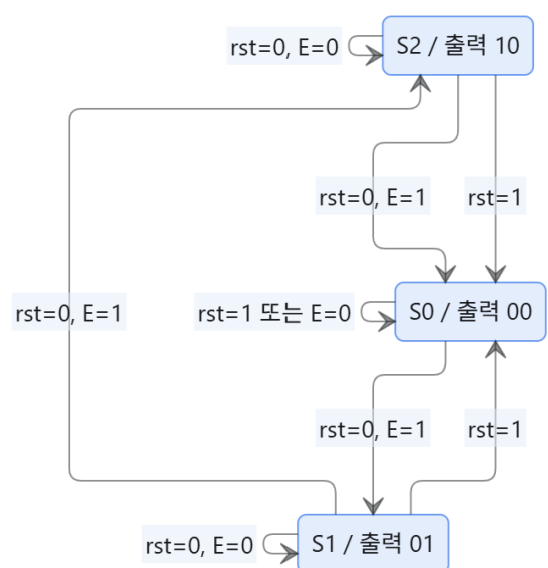
Advance=1, 이전 상태: 10->현재 상태: 00



Rst(1번) 버튼 누른 직후->현재 상태: 00



다음은 무어 머신의 상태도다. 이 때
E=advance and enable이다.



8. 실험7: Mealy 상태 머신

설계와 예상 동작 요약

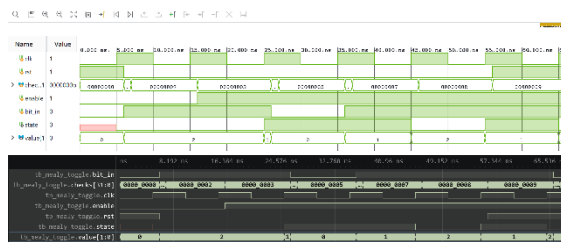
본 회로는 출력이 현재 상태와 입력이 할당된 스위치에 영향을 받는 회로다. 현재 상태는 enable 버튼에 의해서 변화한다.

Vivado 시뮬레이션 및 사전 결과 비교

```
LAB2_PASS mealy_toggle checks=11
$finish called at time : 67 ns :
```

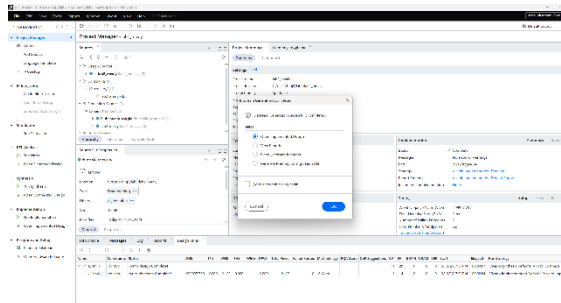
```
LAB2_PASS mealy_toggle checks=11
C:\Users\lab2\OneDrive\Documents\lab2_07_mealy\src\lab2_07_mealy_toggle.v:19: $finish called at 67000 (ns)
```

vivado에서 simulation 한 결과 checks=11, 67ns에서 종료된 것을 확인할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.



위 사진은 vivado simulation의 파형, 아래는 vscode에서 사전 검증한 파형이다. 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



Setup	Hold	Pulse Width
Worst Negative Slack (NNS) 0.00000000 ns	Worst Hold Slack (HNS) 0.00000000 ns	Worst Pulse Width Slack (PWS) 0.00000000 ns
Total Negative Slack (TNS) 0.0000 ns	Total Hold Slack (THS) 0.0000 ns	Total Pulse Width Negative Slack (TPWS) 0.0000 ns
Number of Failing Endpoints 0	Number of Failing Endpoints 0	Number of Failing Endpoints 0
Total Number of Endpoints 22	Total Number of Endpoints 22	Total Number of Endpoints 15

All user specified timing constraints are met.

DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

실제 보드에서는 1번 dip switch를 bit_in, LED 6번을 현재 상태, LED 7,8번을 출력으로 설정하였다. LED 8번은 현재 상태가 1, bit_in이 1이어야만 점등되는 것을 확인하였다. 이 때 상태를 변화시키는 2번 버튼을 누르거나 1번 스위치를 off 했을 때는 소등되는 것을 확인하였다. 리셋 버튼을 누르면 모든 LED가 소등되는 것을 확인하였다.

9. 실험8: 7세그먼트 자리 스캔

설계와 예상 동작 요약

본 회로는 총 8자리의 세그먼트 숫자를 보여주는 회로다. 이 때 세그먼트 숫자는 입력 주파수에 맞게 깜빡이며, 각 자리의 위상은 모두 다르다. 또한, 가장 앞자리의 세그먼트 숫자 및 문자는 스위치를 통해 변경할 수 있다.

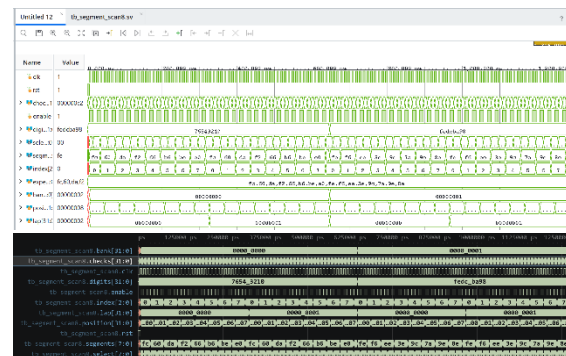
Vivado 시뮬레이션 및 사전 결과 비교

```
LAB2_PASS segment_scan8 checks=194
$finish called at time : 1306 ns :
```

```
LAB2_PASS segment_scan8 checks=194
C:\Users\lab2\OneDrive\Documents\lab2_08_segment_scan\src\lab2_08_segment_scan.v:19: $finish called at 130000 (ns)
```

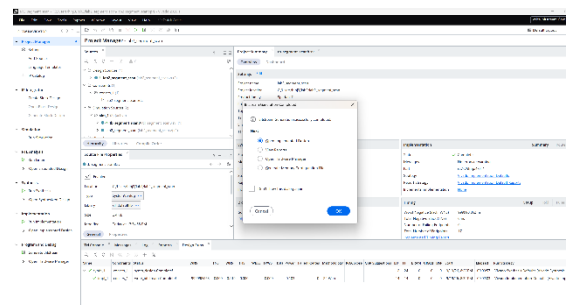
vivado에서 simulation 한 결과 checks=194, 1306ns에서 종료된 것을 확인

할 수 있다. 이는 vscode에서 simulation한 결과와 같음을 확인할 수 있다.



위 사진은 vivado simulation의 파형, 아래는 vscode에서 사전 검증한 파형이다. 위 사진과 아래 사진의 결과가 일치하는 것을 확인할 수 있다.

합성, 구현 결과 및 문제 해결



Synthesis, Implementation 및 Generate bitstream이 전부 성공적으로 실행된 것을 확인할 수 있다.



DRC에는 실험 1과 동일한 CFGBVS-1 경고 1건이 남아 있으며, 구성 전압 관련 속성의 확인과 설정이 필요하다.

실제 보드 구현 및 동작 분석

실제 보드에서는 8 array 7 segment로 출력을 확인하였다. Dip switch 1~4번의

binary 값에 따라 가장 앞자리의 세그먼트 숫자 및 문자가 변화하는 것을 확인하였다. 또한 보드의 입력 주파수를 변경시키자 각 세그먼트 자리의 주파수가 변경되어 깜빡이는 것을 확인하였다. 이 때 입력 주파수가 현저히 느려지면 각 자리를 순차적으로 선택하여 표시하고 자리 선택 사이에 소등 구간을 두기 때문에 각 세그먼트 자리가 왼쪽에서 오른쪽으로 물결치며 점등되는 것처럼 관찰되는 것을 확인하였다.

10.종합 고찰 및 결론

종합 고찰

본 실험에서는 업/다운 카운터, 클록 분주기, 레지스터, 시프트 레지스터, PISO, Moore 및 Mealy 상태 머신과 7세그먼트 자리 스캔 회로를 설계하고 시뮬레이션과 fpga 보드 구현을 통해 순차논리회로의 동작을 검토하였다. 지난 실험에서 설계했던 조합논리회로와 달리 순차논리회로에서는 현재 입력뿐만 아니라 저장된 상태와 클록 시점이 동작에 영향을 준다는 것을 확인했다. 특히 Moore와 Mealy 상태 머신을 비교하면서 출력이 현재 상태에만 의존하는 경우와 현재 입력에도 의존하는 경우의 차이를 이해할 수 있었다.

시뮬레이션에서는 클록 에지 전후의 입력과 상태 변화를 상세히 확인할 수 있었지만, 실제 보드 구현에서는 클록 에지에 따른 변화를 눈으로 구분하는 데에는 한계가 있었다. 또한, 입출력과 연결된 버튼 및 스위치 버튼과 LED에 대해 정확히 파악하는 것이 중요하였다.

결론

본 실험을 통해 여러 순차논리회로를 RTL로 설계하고, 시뮬레이션, 합성, 구현, 비트스트림 생성 및 FPGA 보드 위 구현으로 이어지는 설계 과정을 수행하였다. 특히 실험6 Moore 상태 머신의 시연을 통해 상태 전이 조건과 출력의 관계를 실제 스위치 버튼 조작 및 LED 표시와 연결하여 확인하였다.

11.참고 문헌

[1] M. Morris Mano, Charles R. Kime, Tom Martin, 『디지털논리와 컴퓨터설계』, 제5판, 양세양·최명렬·최석림·하경재 역, 퍼스트북, 2016, pp. 200